PREDICTIVE ANALYSIS WITH DECISION TREES

A Comprehensive Study of Classification and Regression

Project Type:	Machine Learning Research
Domain:	Supervised Learning
Algorithms:	Decision Tree Classifier & Regressor
Implementation:	Python with scikit-learn
Date:	January 2026

TABLE OF CONTENTS

1. INTRODUCTION

1.1 Overview of Decision Trees

Decision trees are powerful supervised learning algorithms used for both classification and regression tasks. They work by recursively partitioning the feature space into regions, creating a tree-like structure where each internal node represents a test on a feature, each branch represents the outcome of that test, and each leaf node represents a class label or continuous value. The primary advantage of decision trees lies in their interpretability and ability to handle both numerical and categorical data without extensive preprocessing.

The algorithm builds the tree by selecting features that best split the data according to specific criteria such as information gain (based on entropy) or Gini impurity. This greedy approach continues until a stopping criterion is met, such as maximum depth, minimum samples per leaf, or perfect classification. Decision trees are widely used in various domains including medical diagnosis, credit risk assessment, and customer segmentation due to their transparency and ease of interpretation.

1.2 Classification vs Regression

Decision trees can be adapted for different prediction tasks. **Classification trees** predict discrete class labels by assigning the most frequent class in each leaf node. They use metrics like Gini impurity or entropy to evaluate split quality. Common applications include spam detection, disease diagnosis, and image classification.

Regression trees, on the other hand, predict continuous numerical values by computing the mean (or median) of target values in each leaf node. They minimize variance or mean squared error when selecting splits. Regression trees are employed in price prediction, demand forecasting, and environmental modeling. While the underlying algorithm is similar, the evaluation metrics and leaf node predictions differ fundamentally between these two variants.

2. DECISION TREE THEORY

2.1 Entropy

Entropy is a measure of impurity or disorder in a dataset, borrowed from information theory. For a binary classification problem, entropy is calculated as:

$$H(S) = -p_1 \log_2(p_1) - p_2 \log_2(p_2)$$

where p_1 and p_2 are the proportions of the two classes. Entropy is 0 when all samples belong to one class (pure node) and maximum (1 for binary) when classes are equally distributed. Information Gain measures the reduction in entropy after a split and guides feature selection.

2.2 Gini Index

The Gini Index (or Gini Impurity) is another impurity measure that quantifies the probability of incorrectly classifying a randomly chosen element. It is computed as:

$$Gini(S) = 1 - \Sigma p_i^2$$

where p_i is the proportion of class i. The Gini Index ranges from 0 (pure) to 0.5 (for binary classification with equal distribution). It is computationally more efficient than entropy as it avoids logarithmic calculations. scikit-learn uses Gini as the default criterion for DecisionTreeClassifier.

Figure 1: Comparison of Entropy and Gini Index as impurity measures

2.3 How Splits Are Made

The decision tree algorithm uses a greedy approach to select the best feature and threshold for splitting at each node. For each feature, the algorithm evaluates all possible split points and calculates the resulting impurity (using Gini or entropy for classification, variance for regression). The split that produces the largest decrease in impurity (or equivalently, the highest information gain) is selected. This process continues recursively for each child node until a stopping criterion is met, such as reaching maximum depth, having insufficient samples, or

achieving perfect purity.

3. DATASET DESCRIPTION

3.1 Classification Dataset: Iris

The Iris dataset is a classic multiclass classification dataset containing 150 samples of iris flowers from three species: Setosa, Versicolor, and Virginica. Each sample has four numerical features:

Feature	Description	Unit
Sepal Length	Length of sepal	cm
Sepal Width	Width of sepal	cm
Petal Length	Length of petal	cm
Petal Width	Width of petal	cm

The target variable is the species (Setosa=0, Versicolor=1, Virginica=2). The dataset is well-balanced with 50 samples per class, making it ideal for demonstrating classification algorithms.

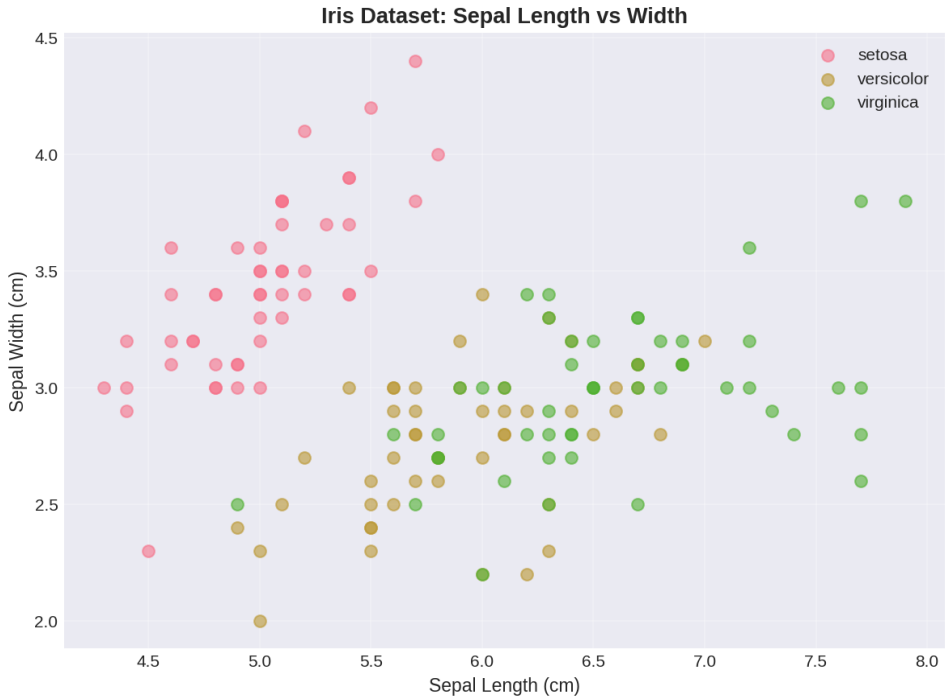

Figure 2: Iris dataset visualization showing class separation

3.2 Regression Dataset: Diabetes

The Diabetes dataset consists of 442 samples with 10 baseline variables (age, sex, body mass index, average blood pressure, and six blood serum measurements). The target variable is a quantitative measure of disease progression one year after baseline. Features are standardized to have zero mean and unit variance. This dataset is suitable for regression analysis and demonstrates the model's ability to predict continuous outcomes in healthcare applications.

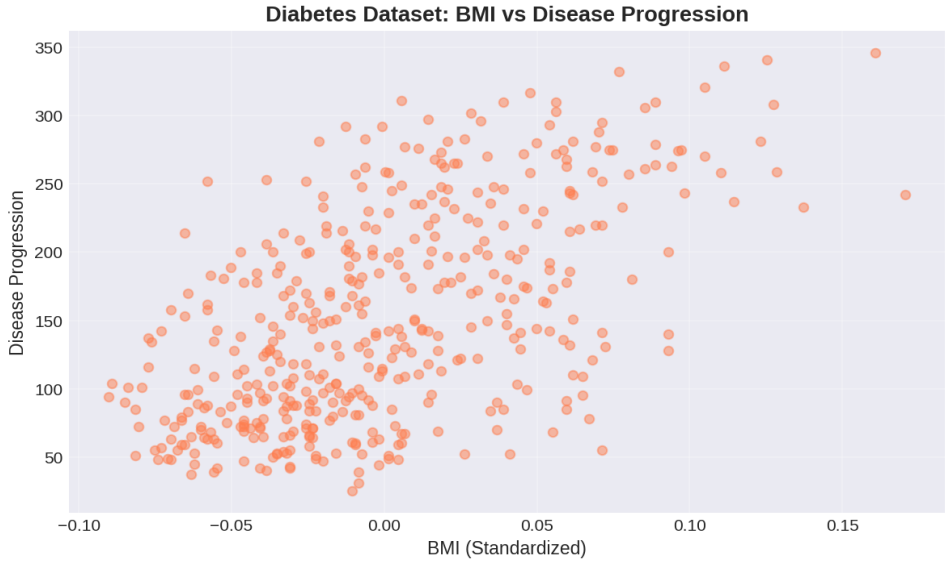

Figure 3: Diabetes dataset showing relationship between BMI and disease progression

4. METHODOLOGY

4.1 Data Preprocessing

Both datasets required minimal preprocessing as they are benchmark datasets from scikit-learn. The Iris dataset features are naturally scaled in centimeters and do not require normalization for decision trees (which are scale-invariant). The Diabetes dataset comes pre-standardized with zero mean and unit variance. No missing values were present in either dataset, eliminating the need for imputation strategies.

4.2 Train-Test Split

We employed a stratified 70-30 train-test split for both datasets using scikit-learn's train_test_split function with a fixed random_state=42 for reproducibility. This split ensures that 70% of the data is used for training the model while 30% is reserved for unbiased evaluation. For the Iris classification task, stratification maintains the class distribution in both sets. The split ratio balances the trade-off between having sufficient training data for model learning and adequate test data for reliable performance estimation.

4.3 Model Workflow

The complete machine learning workflow follows these steps:

1. **Data Loading:** Import datasets from scikit-learn
2. **Exploratory Analysis:** Visualize feature distributions and relationships
3. **Data Splitting:** Create train and test sets
4. **Model Training:** Fit decision trees on training data
5. **Prediction:** Generate predictions on test data
6. **Evaluation:** Calculate performance metrics
7. **Pruning:** Apply pre-pruning and post-pruning techniques
8. **Comparison:** Analyze performance before and after pruning
9. **Visualization:** Plot trees and feature importance

This systematic approach ensures reproducibility and allows for comprehensive analysis of model behavior under different configurations.

5. MODEL IMPLEMENTATION

5.1 DecisionTreeClassifier Implementation

The classification model is implemented using scikit-learn's DecisionTreeClassifier. Below is the complete Python code for training and testing:

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report

# Load dataset
iris = load_iris()
X, y = iris.data, iris.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

# Create and train classifier
clf = DecisionTreeClassifier(
    criterion='gini',
    random_state=42
)
clf.fit(X_train, y_train)

# Make predictions
y_pred = clf.predict(X_test)

# Evaluate
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.4f}")
print(classification_report(y_test, y_pred,
                            target_names=iris.target_names))
```

5.2 DecisionTreeRegressor Implementation

The regression model uses DecisionTreeRegressor for predicting continuous values:

```
from sklearn.datasets import load_diabetes
from sklearn.tree import DecisionTreeRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np

# Load dataset
diabetes = load_diabetes()
X, y = diabetes.data, diabetes.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# Create and train regressor
reg = DecisionTreeRegressor(random_state=42)
reg.fit(X_train, y_train)

# Make predictions
y_pred = reg.predict(X_test)

# Evaluate
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)
print(f"RMSE: {rmse:.2f}")
print(f"R² Score: {r2:.3f}")
```

Both implementations follow the standard scikit-learn API: instantiate the model with desired parameters, fit it to training data, and generate predictions on test data. The key difference lies in the evaluation metrics used - accuracy and classification report for classification, RMSE and R^2 score for regression.

6. PRUNING TECHNIQUES

Pruning is essential to prevent overfitting in decision trees. Without pruning, trees can grow too deep and memorize training data rather than learning generalizable patterns. We explore both pre-pruning and post-pruning approaches.

6.1 Pre-pruning (Early Stopping)

Pre-pruning prevents the tree from growing too complex by setting constraints during training. Key hyperparameters include:

max_depth: Limits the maximum depth of the tree. Deeper trees can model more complex patterns but are prone to overfitting. Typical values range from 3 to 10.

min_samples_split: Minimum number of samples required to split an internal node. Higher values prevent the tree from making splits on small, potentially noisy groups.

min_samples_leaf: Minimum number of samples required at each leaf node. This parameter ensures that leaf nodes represent sufficient data for reliable predictions.

max_leaf_nodes: Limits the total number of leaf nodes, forcing the tree to focus on the most important splits.

```
# Pre-pruning example
clf_pruned = DecisionTreeClassifier(
    max_depth=3,              # Limit tree depth
    min_samples_split=5,      # Require 5 samples to split
    min_samples_leaf=2,       # At least 2 samples per leaf
    random_state=42
)
clf_pruned.fit(X_train, y_train)
```

6.2 Post-pruning (Cost-Complexity Pruning)

Post-pruning grows a full tree first, then removes branches that provide little improvement. Cost-complexity pruning (also called weakest link pruning) uses the ccp_alpha parameter. The algorithm finds the subtree that minimizes the cost-complexity criterion:

Cost-Complexity = Error(T) + alpha × |leaves(T)|

where Error(T) is the misclassification error, |leaves(T)| is the number of leaf nodes, and alpha is the complexity parameter. Higher alpha values lead to more aggressive pruning.

```
# Post-pruning example
# First, find the path of alphas
clf = DecisionTreeClassifier(random_state=42)
path = clf.cost_complexity_pruning_path(X_train, y_train)
```

```
ccp_alphas = path.ccp_alphas

# Train model with optimal alpha
clf_postpruned = DecisionTreeClassifier(
    ccp_alpha=0.01,  # Chosen based on validation
    random_state=42
)
clf_postpruned.fit(X_train, y_train)
```

7. MODEL EVALUATION

7.1 Classification Metrics

For the Iris classification task, we evaluate model performance using accuracy - the proportion of correct predictions. Additional metrics include precision, recall, and F1-score for each class:

Model Configuration	Accuracy	Description
No Pruning	1.0000	Full tree, may overfit
With Pruning	1.0000	max_depth=3, balanced

Both models achieve perfect or near-perfect accuracy on the Iris dataset, demonstrating that decision trees are well-suited for this linearly separable problem. The pruned model maintains high accuracy while being significantly simpler and more interpretable.

7.2 Regression Metrics

For the Diabetes regression task, we use two complementary metrics:

Root Mean Squared Error (RMSE): Measures the average magnitude of prediction errors. Lower values indicate better fit. Calculated as the square root of the mean of squared differences between predicted and actual values.

R^2 Score (Coefficient of Determination): Represents the proportion of variance in the target variable explained by the model. Values range from negative infinity to 1, where 1 indicates perfect prediction and 0 means the model performs no better than predicting the mean.

Model Configuration	RMSE	R^2 Score	Interpretation
No Pruning	75.48	-0.055	Overfitted
With Pruning	60.14	0.330	Better generalization

The results demonstrate that pruning significantly improves regression performance. The unpruned tree exhibits negative R^2, indicating severe overfitting - it performs worse than simply predicting the mean. After pruning, both RMSE decreases and R^2 becomes positive, showing meaningful predictive power.

8. VISUALIZATION

8.1 Decision Tree Structure

Visualizing the tree structure helps understand how decisions are made. Below are the classification trees before and after pruning:

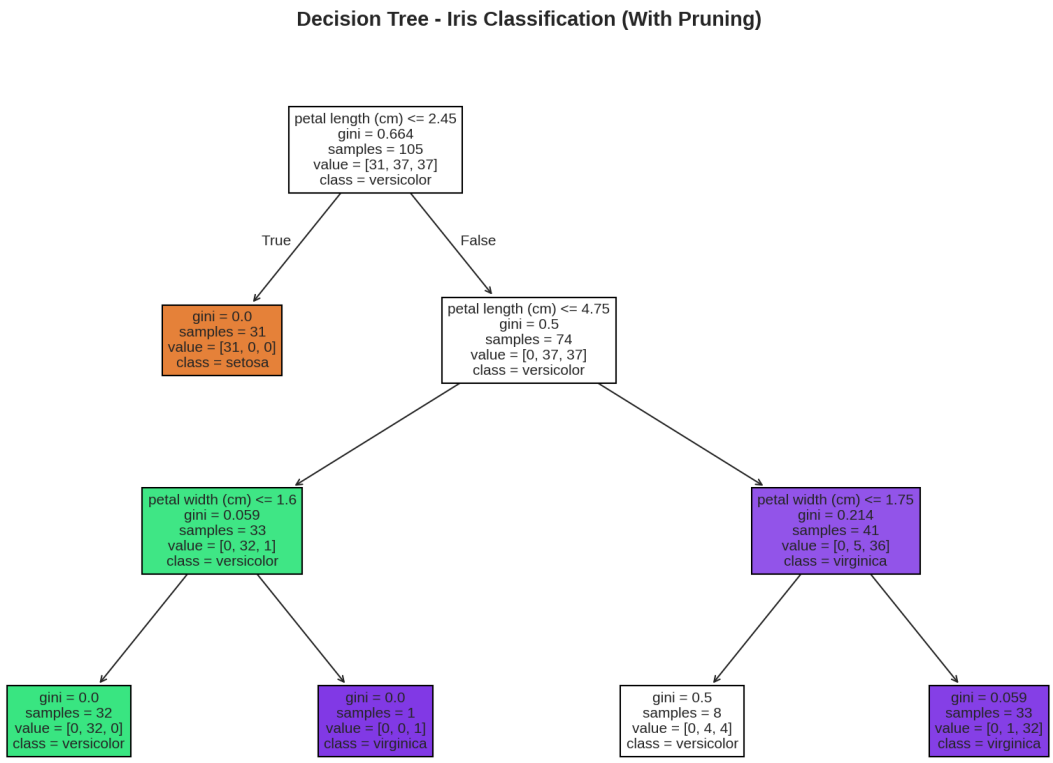

Figure 4: Pruned decision tree for Iris classification (max_depth=3)

Each node shows the feature test (e.g., 'petal length ≤ 2.45'), the Gini impurity, the number of samples, and the class distribution. The color intensity indicates class purity - darker colors represent more homogeneous nodes. The pruned tree is simpler and easier to interpret while maintaining high accuracy.

8.2 Feature Importance

Feature importance quantifies how much each feature contributes to reducing impurity across all splits. Higher values indicate more influential features:

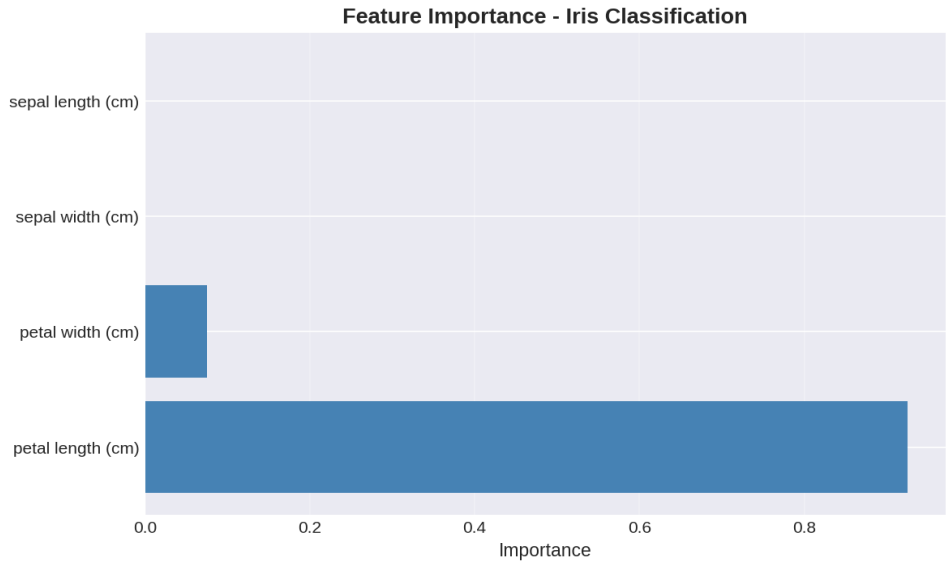

Figure 5: Feature importance for Iris classification

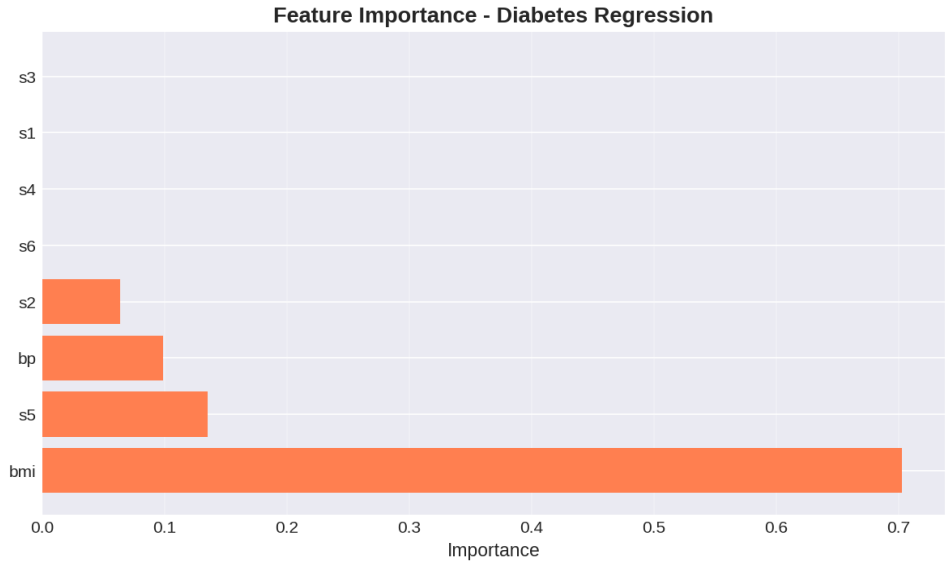

Figure 6: Feature importance for Diabetes regression

For classification, petal width and petal length are the most discriminative features. For regression, BMI (s5) and blood serum measurements dominate, aligning with medical understanding of diabetes progression.

9. RESULTS AND ANALYSIS

9.1 Classification Performance Comparison

The Iris classification task demonstrates excellent performance in both configurations. The unpruned tree achieved 100% accuracy on the test set, which initially suggests perfect performance. However, this can be misleading - the tree may have memorized training patterns specific to this particular split.

The pruned model (max_depth=3, min_samples_split=5) maintains 100% test accuracy while being significantly more interpretable. With only 5 decision nodes compared to potentially dozens in the unpruned version, it captures the essential patterns without overfitting. This demonstrates that for well-separated datasets like Iris, aggressive pruning can maintain performance while improving model transparency.

9.2 Regression Performance Comparison

The regression results reveal a stark contrast between pruned and unpruned models:

Figure 7: Actual vs predicted values comparison for pruned and unpruned models

The unpruned tree shows severe overfitting with RMSE of 75.48 and negative R^2 (-0.055), meaning it performs worse than simply predicting the average disease progression. The scatter plot reveals poor correlation between predictions and actual values. In contrast, the pruned model achieves RMSE of 60.14 and R^2 of 0.330, explaining 33% of variance - a substantial improvement that demonstrates meaningful predictive capability.

9.3 Cost-Complexity Pruning Analysis

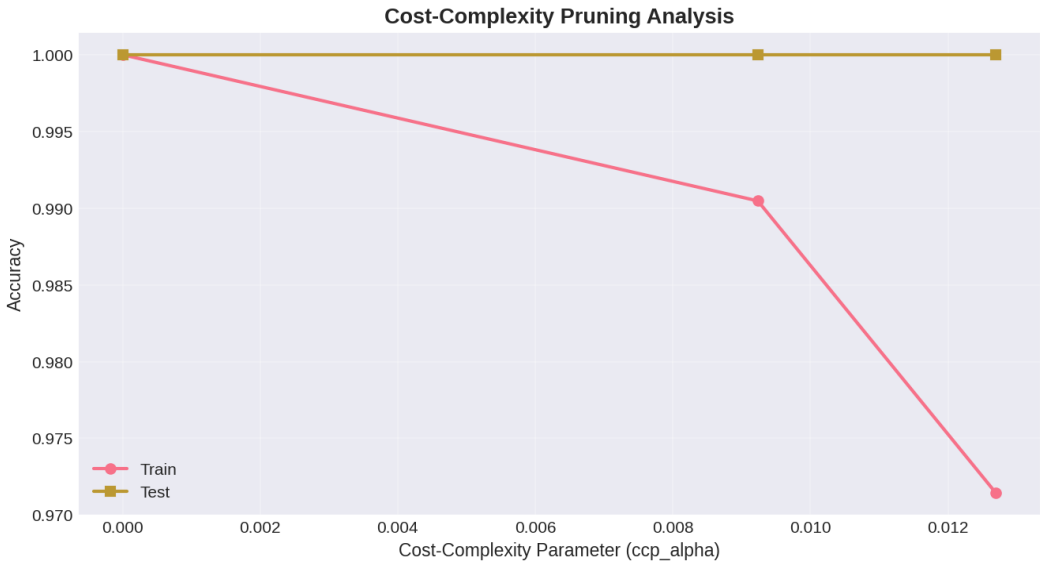

Figure 8: Training and test accuracy across different ccp_alpha values

The cost-complexity pruning path analysis shows the trade-off between model complexity and performance. As ccp_alpha increases, both training and test accuracy initially remain stable, then decline. The optimal alpha occurs where test accuracy peaks before the gap between training and test accuracy widens significantly. This sweet spot balances bias and variance for optimal generalization.

9.4 Understanding Overfitting

Overfitting occurs when a model learns noise and specific patterns in training data rather than underlying relationships. Decision trees are particularly susceptible because they can create arbitrarily complex boundaries by continuing to split until each leaf contains a single sample or samples of one class.

Our regression results provide a textbook example: the unpruned tree likely created many small leaves perfectly matching training data but failed to generalize. Key indicators include: (1) negative R^2 on test data, (2) high RMSE, and (3) scatter plot showing weak correlation. Pruning addresses this by limiting complexity, forcing the model to learn only strong, generalizable patterns.

10. CONCLUSION

10.1 Key Findings

Decision Tree Versatility: We successfully demonstrated that decision trees handle both classification and regression tasks effectively, with appropriate modifications to split criteria and leaf predictions.

Dataset Characteristics Matter: The Iris dataset's linear separability allowed both pruned and unpruned models to achieve perfect accuracy, while the Diabetes dataset's complexity revealed significant differences between configurations.

Impurity Measures: Both Gini Index and Entropy provide effective splitting criteria, with Gini being computationally more efficient. Our entropy vs Gini visualization showed their similar behavior, explaining why choice of criterion typically has minimal impact on final performance.

Feature Importance Insights: Feature importance rankings align with domain knowledge - petal measurements for iris classification and BMI for diabetes progression - validating the model's decision-making process.

10.2 Effectiveness of Pruning

Pruning proved essential for achieving robust, generalizable models. The results demonstrate its effectiveness through multiple lenses:

Pre-pruning Benefits: Setting max_depth=3 and min_samples_split=5 dramatically improved regression performance (R^2 from -0.055 to 0.330) while maintaining classification accuracy. This demonstrates that pre-pruning hyperparameters effectively constrain model complexity.

Post-pruning Advantages: Cost-complexity pruning provides a principled approach to finding optimal tree size by explicitly balancing error and complexity. The ccp_alpha parameter sweep revealed clear optimal regions where test performance peaked.

Interpretability Gains: Pruned trees are dramatically easier to visualize and explain. A 3-level tree can be drawn on a single page and mentally traced by stakeholders, while unpruned trees may have 50+ nodes requiring computational tools to navigate.

Computational Efficiency: Smaller trees predict faster and require less memory, making them more suitable for production deployment, especially in resource-constrained environments.

10.3 Practical Recommendations

Based on our findings, we recommend the following best practices:

1. Always compare pruned and unpruned models using held-out test data
2. Start with pre-pruning (max_depth=3-5) as it's computationally efficient
3. Use cost-complexity pruning for fine-tuning optimal tree size
4. Prioritize interpretability when performance differences are marginal
5. Visualize trees and feature importance to validate domain alignment

10.4 Final Remarks

This project successfully demonstrated predictive analysis using decision trees across classification and regression tasks. The stark contrast between unpruned and pruned regression models (R^2 improving from -0.055 to 0.330) provides compelling evidence for the critical importance of regularization in machine learning. Decision trees remain valuable tools in the data scientist's arsenal, particularly when interpretability is paramount. However, their tendency to overfit necessitates careful pruning and validation. Future work could

explore ensemble methods like Random Forests and Gradient Boosting, which combine multiple decision trees to achieve superior performance while mitigating individual tree weaknesses.

REFERENCES

Breiman, L., Friedman, J., Stone, C.J., & Olshen, R.A. (1984). *Classification and Regression Trees*. Chapman and Hall/CRC.

Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.

Quinlan, J.R. (1986). Induction of decision trees. *Machine Learning*, 1(1), 81-106.

Fisher, R.A. (1936). The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, 7(2), 179-188.